



University of Antwerp
| Faculty of Applied
Engineering

Lab 4 – Discovery

Distributed Systems

Andreas Gavrielides

University of Antwerp – imec, IDLab – Faculty of Applied Engineering, Belgium

Overview of lab sessions

- **Lab 1: DONE**
 - Introductory lab & TCP/UDP
 - Setup Remote nodes
- **Lab 2: DONE**
 - REST protocol
 - Remote nodes
 - 2 sessions
- **Lab 3: Project -> in progress**
 - *Naming server*
 - *Remote nodes*
 - *2 sessions*
- **Lab 4: Project -> Today**
 - Discovery service
 - Remote nodes
 - 2 sessions
- **Lab 5: Project**
 - Replication
 - Remote nodes
 - 3 sessions
- **Lab 6: Project**
 - Agents
 - Remote nodes
 - 2 sessions
- **Lab 7: Project**
 - GUI
 - Remote nodes
 - 1 session
- **Finishing project**
 - Remote nodes
 - 5 sessions

Lab Discovery [project]

- **Goal of the session:**

- To enable `node life-cycle` in system Y

- **Team work:**

- Work in your groups (available on Blackboard), **divide the work!**

- **Node life-cycle:**

- Discovery
- Bootstrap
- Shutdown
- Failure

Node life-cycle operations

- **Discovery:**

- Auto-discover the Naming server and existing nodes in the network

- **Bootstrap:**

- Starting each node, initializing local parameters (previous, next node), and updating parameters of existing nodes

- **Shutdown:**

- A node leaves the ring network of system Y, and updates parameters of neighbour nodes and the Naming server

- **Failure:**

- Node's failure is detected. Naming server updates parameters of remaining nodes

Discovery and Bootstrap (1/2)

1. Develop a method that sends multicast message to existing nodes and the Naming server (i.e., all nodes in the local network). This method will be needed in step 3
2. Develop a method that calculates a hash based on the node name. Try to reuse this functionality from the previous session.
3. During bootstrap the node will send its name and its IP address to all nodes and the Naming server in the network using multicast
4. The Naming server receives the multicast message and executes the following steps:
 1. Calculate the hash of a node name
 2. Adds the hash and IP address in its map data structure (see lab Naming server)
 3. Responds to the new node with the number of existing nodes that are currently in the network

Discovery and Bootstrap (2/2)

5. **Other nodes in the network also receive this multicast message, and they perform the following:**
 1. Calculate the hash of the node that sent the multicast message
 2. Each node calculates its own hash and stores it as `currentID`
 3. If `currentID < hash < nextID`, `nextID = hash`, current node updates its own parameter `nextID`, **and sends response to node** giving the information on `currentID` and `nextID`
 4. If `previousID < hash < currentID`, `previousID = hash`, current node updates its own parameter `previousID`, **and sends response to node** giving the information on `currentID` and `previousID`
6. **A node that sent multicast message receives response from Naming server:**
 1. If the number of existing nodes in the network is < 1 , it means that this is the only node in the network (this node is its previous and next node, `previousID = currentID`, `nextID = currentID`)
 2. If the number of existing nodes in the network is > 1 , this node receives parameters for its previous and next node of corresponding nodes in the network

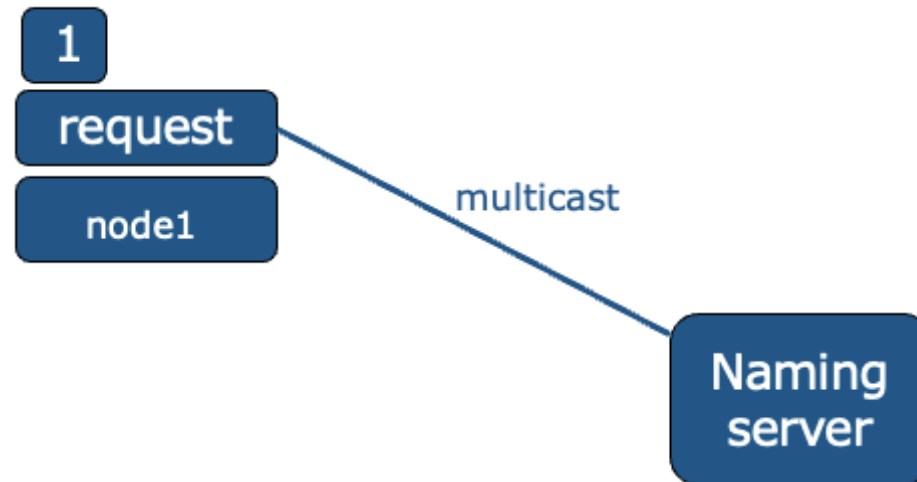
Shutdown

- Send the ID of *the next node* to *the previous node*. In the previous node, the next node parameter will be updated according to this information.
- Send the ID of *the previous node* to *the next node*. In the next node, the previous node parameter will be updated according to this information
- Remove the node from the Naming server's Map

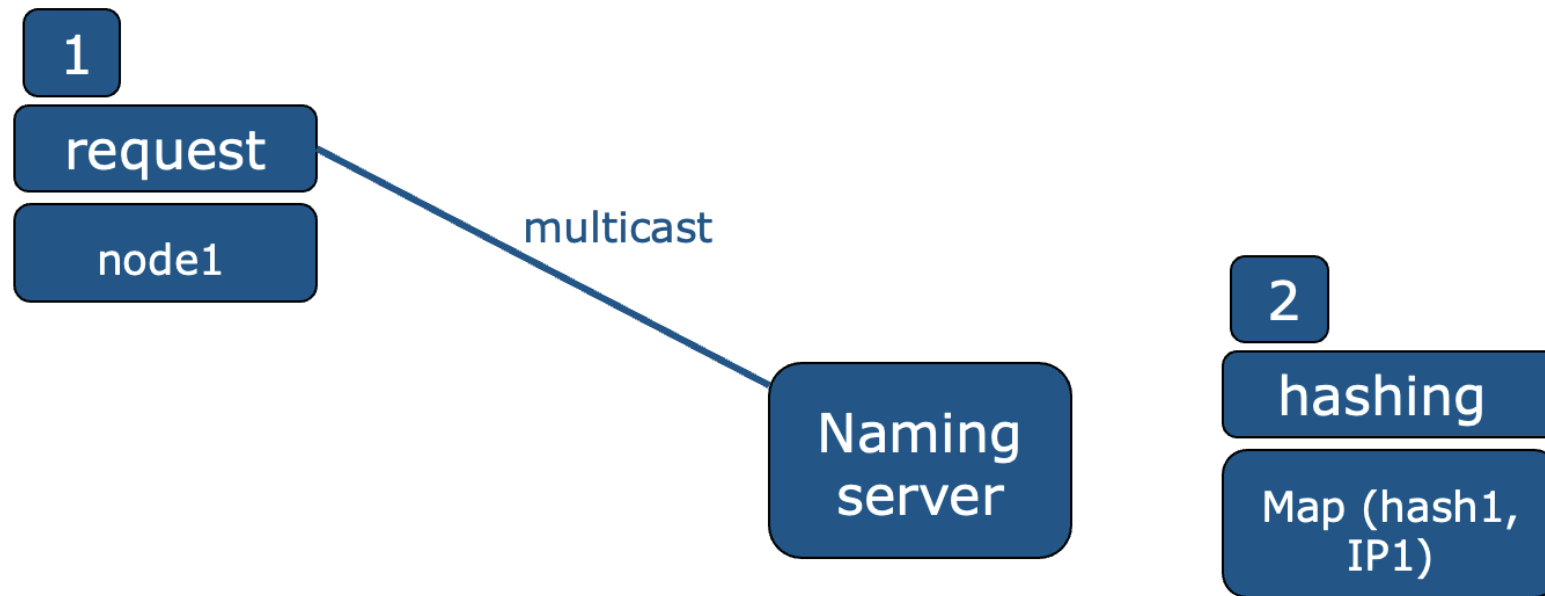
Failure

- This **algorithm** is **activated in every exception** thrown during communication with other nodes. This allows distributed detection of node failure
- Request the previous node and next node parameters from the nameserver
- Update the `next node` parameter of the previous node with the information received from the nameserver
- Update the `previous node` parameter of the next node with the information received from the nameserver
- Remove the node from the Naming server
- Test this algorithm manually terminating a node (CTRL – C) and use a ping method as part of each node, that throws an exception when connection fails to a given node

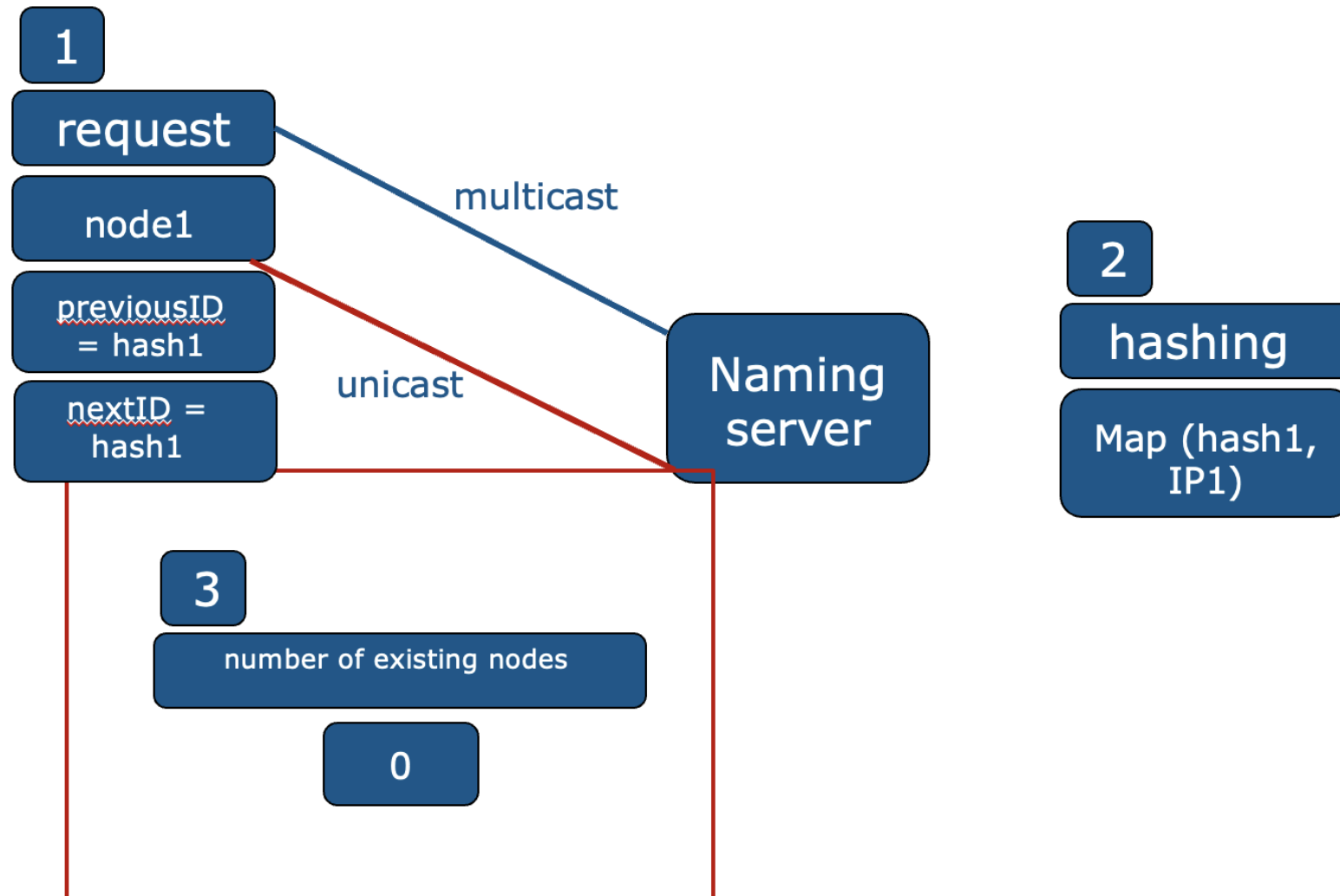
Bootstrap and Discovery: example 1



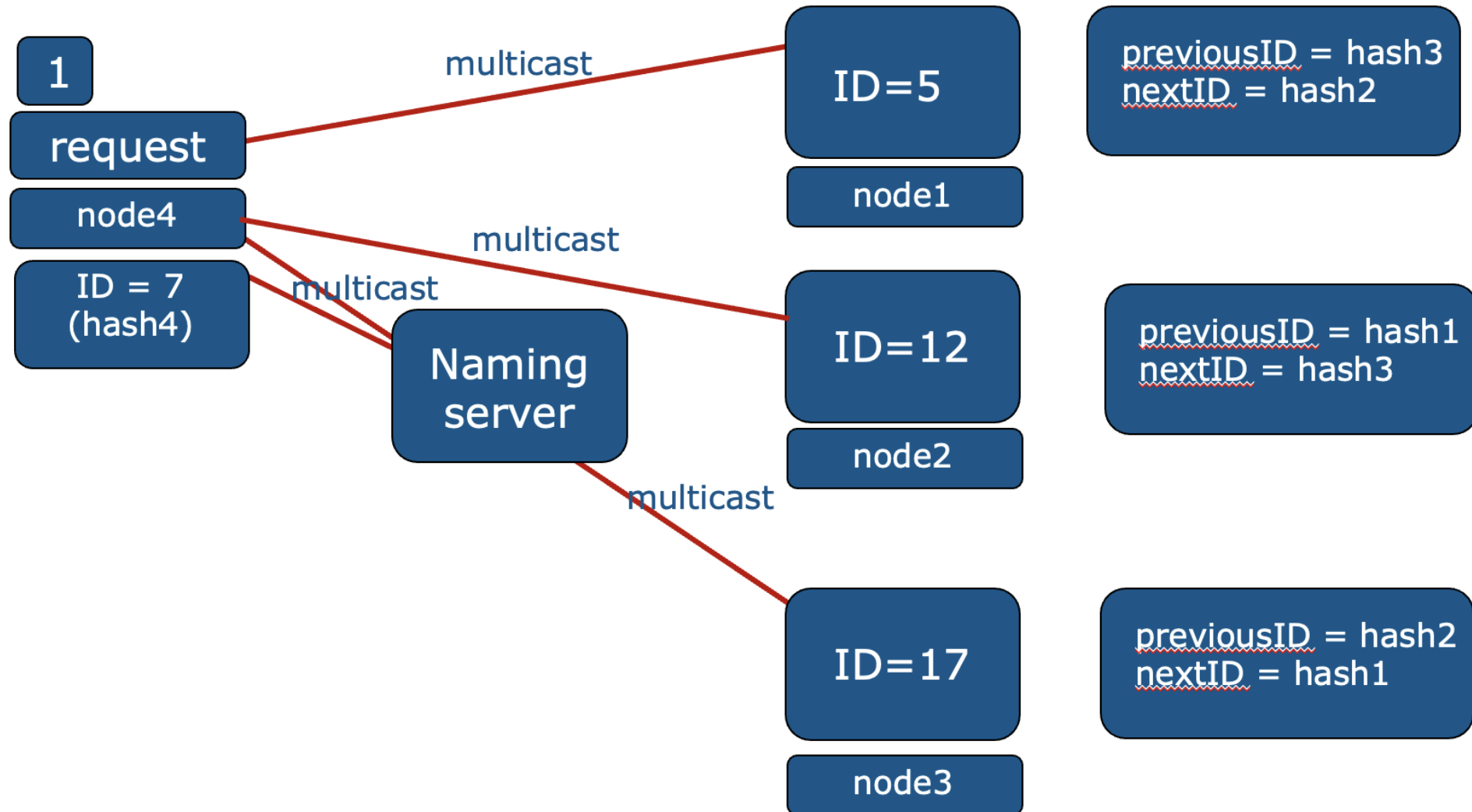
Bootstrap and Discovery: example 1



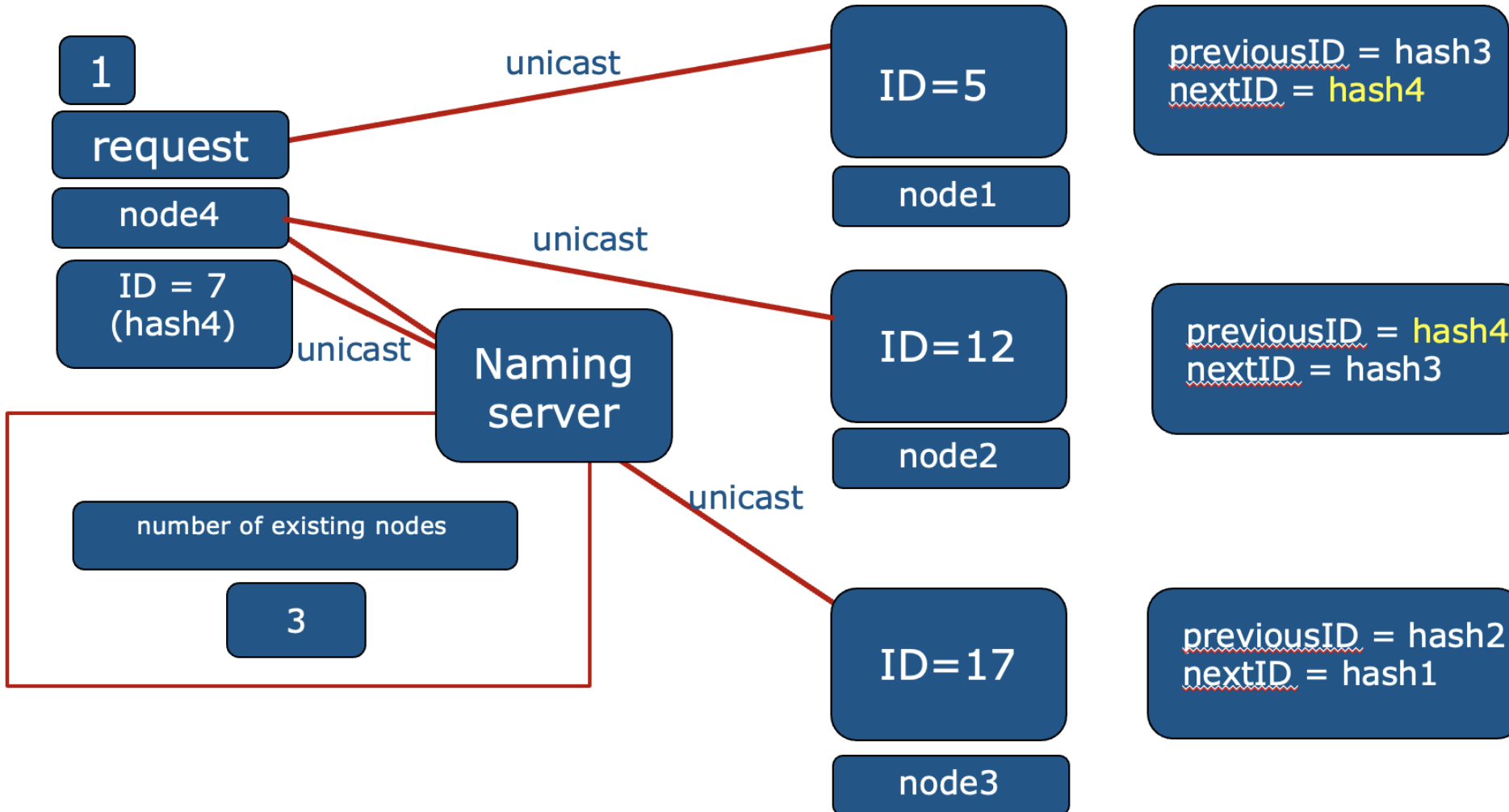
Bootstrap and Discovery: example 1



Bootstrap and Discovery: example 2



Bootstrap and Discovery: example 2



If $\text{currentID} < \text{hash} < \text{nextID}$, $\text{nextID} = \text{hash}$,
 $\text{hash1} < \text{hash4} < \text{hash2}$
 $5 < 7 < 12$
 $\Rightarrow \text{YES nextID} = 7 (= \text{hash4})$

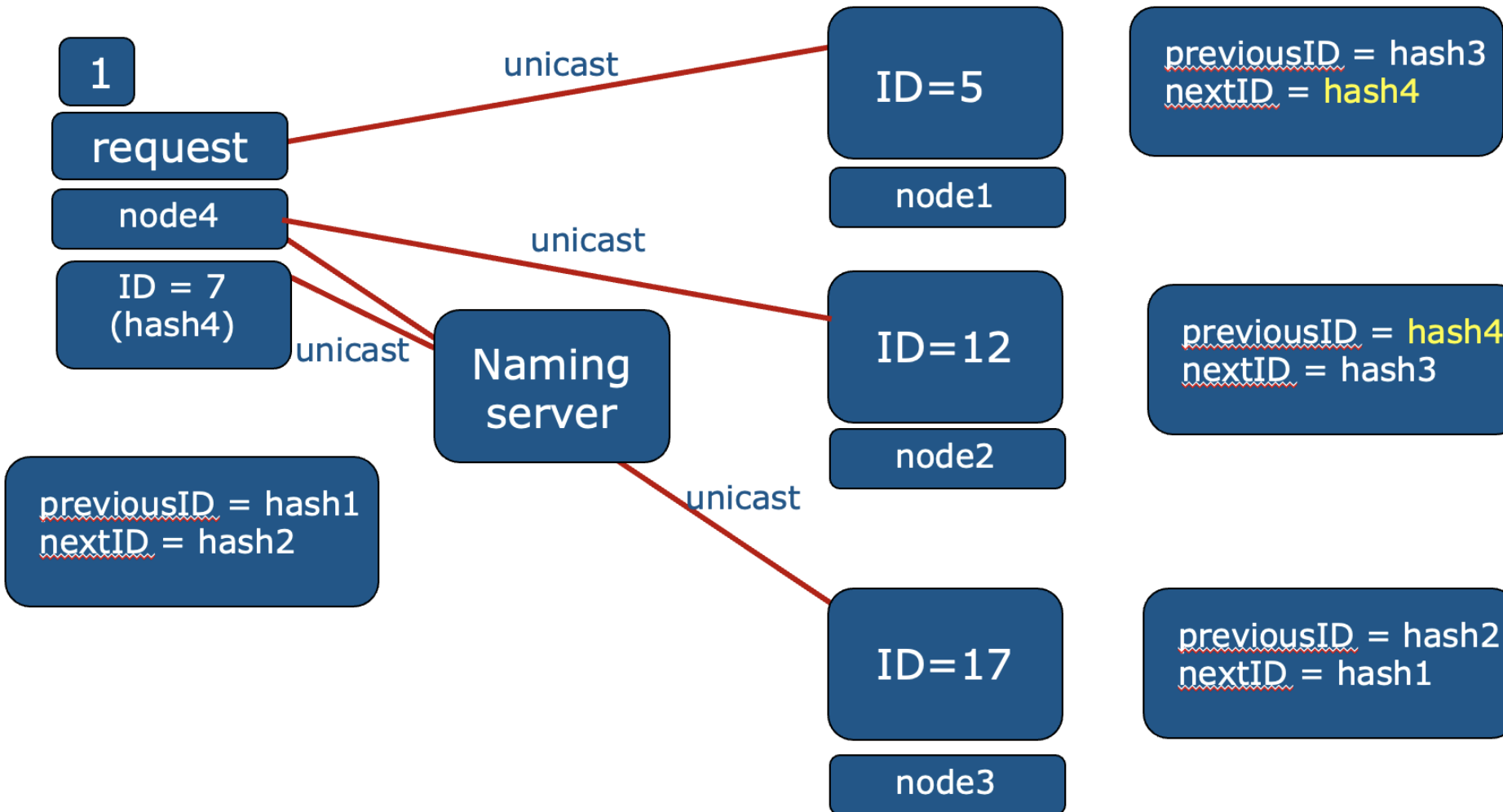
current node updates its own parameter nextID, and sends response to node giving the information on currentID and nextID

If $\text{previousID} < \text{hash} < \text{currentID}$, $\text{previousID} = \text{hash}$,
 $\text{hash1} < \text{hash4} < \text{hash2}$,
 $5 < 7 < 12$
 $\Rightarrow \text{YES previousID} = 7 (= \text{hash4})$

current node updates its own parameter previousID, and sends response to node giving the information on currentID and previousID

previousID = hash2
nextID = hash1

Bootstrap and Discovery: example 2



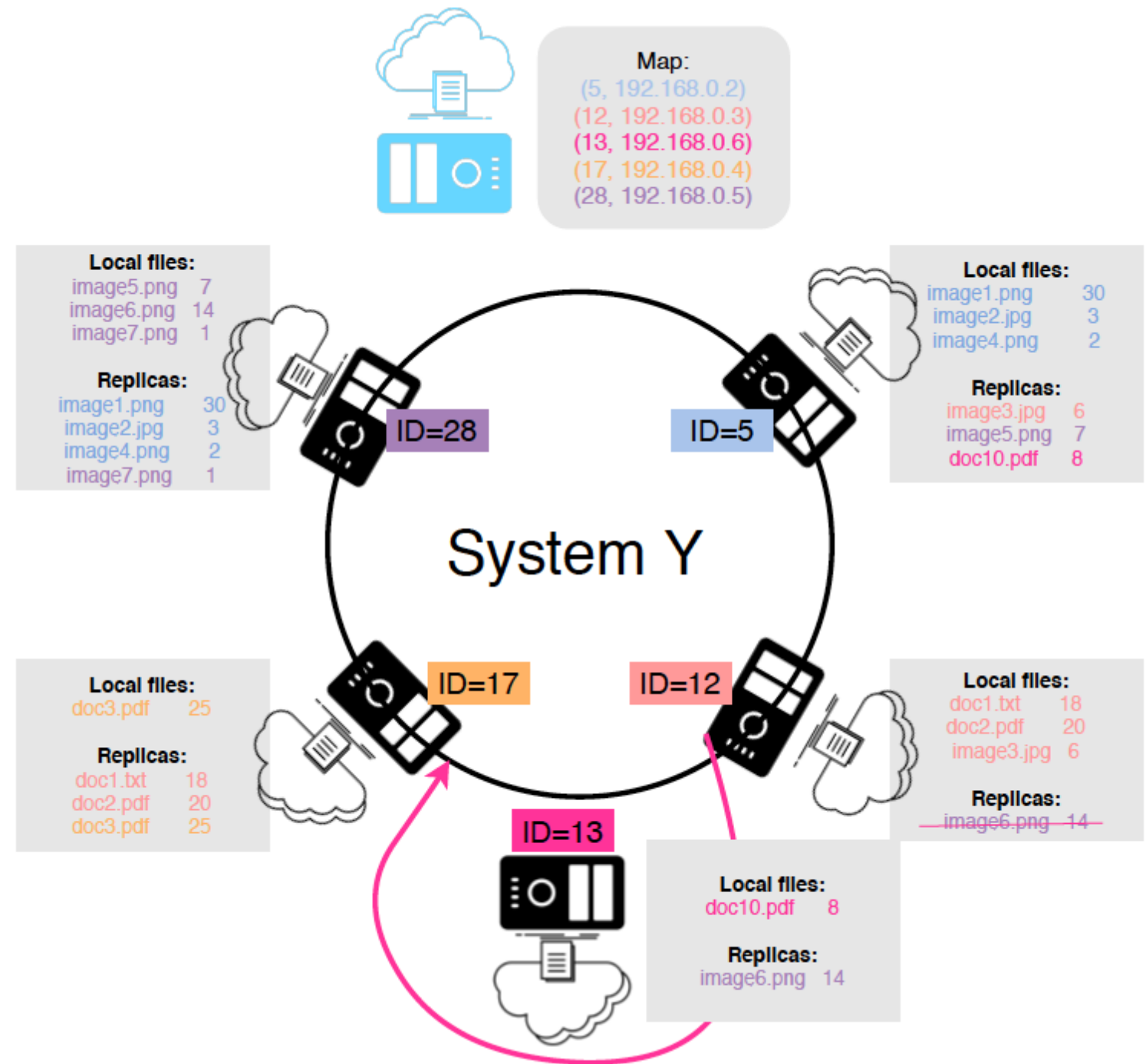
If $\text{currentID} < \text{hash} < \text{nextID}$, $\text{nextID} = \text{hash}$,
 $\text{hash1} < \text{hash4} < \text{hash2}$
 $5 < 7 < 12$
 $\Rightarrow \text{YES nextID} = 7 (= \text{hash4})$

current node updates its own parameter nextID, and sends response to node giving the information on currentID and nextID

If $\text{previousID} < \text{hash} < \text{currentID}$, $\text{previousID} = \text{hash}$,
 $\text{hash1} < \text{hash4} < \text{hash2}$,
 $5 < 7 < 12$
 $\Rightarrow \text{YES previousID} = 7 (= \text{hash4})$

current node updates its own parameter previousID, and sends response to node giving the information on currentID and previousID

Bootstrap and Discovery: example 3



Deadline

- **2 sessions**
 - Deadline report discovery -> 26/04/2026 (23:59)